LXC

LXC (Linux Containers) — механизм виртуализации на уровне операционной системы, позволяющий исполнять множество изолированных Linux-систем (контейнеров) в одной системе.

Ядро Linux может изолировать ресурсы (процессор, память, ввод/вывод, сеть и так далее) при помощи cgroups, не прибегая для этого к использованию виртуальных машин. Посредством cgroups изолируются так же деревья процессов, сеть, пользователи и файловые системы.

LXC комбинирует cgroups и пространства имён (namespace).

На данный момент использует LXC следующие возможности ядра:

- Kernel namespaces (ipc, uts, mount, pid, network and user)
- Apparmor and SELinux profiles
- Seccomp policies
- Chroots (using pivot_root)
- Kernel capabilities
- CGroups (control groups)

LXC используется в разнообразных проектах, в том числе в Docker.

LXC можно воспринимать как что-то среднее между chroot и полноценной виртуальной машиной, такой chroot на стероидах.

1 Установка

- 2 Команды
- 3 Примеры использования
  - 3.1 Создание контейнера.
  - 3.2 Запуск контейнера
  - 3.3 Просмотр запущенных контейнеров
  - 3.4 Просмотр состояния конкретного контейнера
  - 3.5 Подключение к консоли запущенного контейнера
  - 3.6 Остановка контейнера
  - 3.7 Клонирование контейнера
- 4 Настройка сети
  - 4.1 loopback interface (type = empty)
  - 4.2 Использование физического интерфейса (type = phys)
  - 4.3 Использование сетевого стека хост-системы (type = veth)
    - 4.3.1 Трансляция ip-адресов
    - 4.3.2 Статические ip-адреса
    - 4.3.3 Использование DHCP
  - 4.4 Несколько виртуальных сетевых интерфейсов с разными MAC-адресами (type = macvlan)
- 5 Ограничение ресурсов
  - 5.1 Память
  - 5.2 CPU
  - 5.3 Дисковые квоты: использование loopback-устройств
  - 5.4 Запуск контейнеров поверх LVM
  - 5.5 Проверка ограничений
- 6 Проброс устройств
- 7 Конфигурация контейнера
- 8 X-Server
- 9 LXC без LXC
  - 9.1 libvirt
  - 9.2 libcontainer
- 10 LXC в сравнении с другими проектами
  - 10.1 LXC vs OpenVZ
  - 10.2 LXC vs Xen
  - 10.3 LXC vs KVM
- 11 Вопросы и ответы
- 12 Графический интерфейс
- 13 Материалы по виртуализации, паравиртуализации и эмуляции на Xgu.ru

Установка

1. Инсталлируем сразу все необходимые пакеты:

```
sudo apt-get install lxc debootstrap bridge-utils
```

2. Монтирование cgroup. Пакет lxc зависит от пакета cgroup-lite, который монтирует каждую cgroup подсистему отдельно в `/sys/fs/cgroup/` (в Debian и Ubuntu cgroup вручную монтировать не нужно), но если cgroup все же не смонтирован, то:

добавляем строку в `/etc/fstab`:

```
cgroup  /sys/fs/cgroup  cgroup  defaults  0   0
```

монтируем:

```
sudo mount /sys/fs/cgroup
```

В принципе, точка монтирования не важна -- можно создать любой каталог (например, sudo mkdir /cgroup) и сохранить соответствующую запись в `/etc/fstab`:

```
cgroup  /cgroup   cgroup  defaults  0   0
```

3. Проверяем правильность установки:

```
sudo lxc-checkconfig
```

В достаточно длинном выводе команды не должно присутствовать сообщений об ошибках:

```
--- Namespaces ---
Namespaces: enabled
Utsname namespace: enabled
Ipc namespace: enabled
Pid namespace: enabled
User namespace: enabled
Network namespace: enabled
Multiple /dev/pts instances: enabled

--- Control groups ---
Cgroup: enabled
Cgroup clone_children flag: enabled
Cgroup device: enabled
Cgroup sched: enabled
Cgroup cpu account: enabled
Cgroup memory controller: enabled
Cgroup cpuset: enabled

--- Misc ---
Veth pair device: enabled
Macvlan: enabled
Vlan: enabled
File capabilities: enabled
```

Команды

lxc-create
: создать новый контейнер LXC

lxc-start
: запустить контейнер LXC

lxc-console
: подключиться к консоли указанного контейнера

lxc-attach
: запустить указанную программу внутри контейнера - (NOT SUPPORTED) Run a command in a running container

lxc-destroy
lxc-stop
: остановить процесс, работающий внутри контейнера

lxc-execute
: выполнить указанную команду внутри контейнера (в чём отличие от lxc-attach?)

lxc-monitor
: мониторить состояние контейнеров

lxc-wait
: ждать определённого состояния контейнера; завершаться, когда состояние достигнуто

lxc-cgroup
: управление cgroup-группами контейнера

lxc-ls
: показать список контейнеров в системе

lxc-ps
: показать список процессов внутри определённого контейнера

lxc-info
: показать информацию о заданном контейнере

lxc-freeze
: заморозить все процессы указанного контейнера

lxc-unfreeze
: разморозить все процессы указанного контейнера

Примеры использования

Создание контейнера.

Реализуется через выполнение шаблонов, командой: **lxc-create -t <название шаблона> -n <название контейнера>**:

```
sudo lxc-create -t ubuntu -n test_01
```

Файлы доступных шаблонов (для debian) находятся в каталоге `/usr/share/lxc/templates/`

После загрузки и инсталляции пакетов, контейнеры будут размещаться в `/var/lib/lxc/<название контейнера>`. В данном примере -- это каталог `/var/lib/lxc/test_01`, в котором:

```
-rw-r--r-- 1  root root 685  Дек 10 09:52 config
-rw-r--r-- 1  root root 1181 Дек 10 00:02 test_01.log
-rw-r--r-- 1  root root 0    Дек 9  16:04 fstab
drwxr-xr-x 22 root root 4096 Дек 10 10:00 rootfs
```

Файл `config` -- файл конфигурации контейнера, а `rootfs` -- каталог, в котором развернута файловая система ubuntu.

В команду lxc-create можно передать параметры, в том числе желаемую версию дистрибутива. Чтобы узнать, какие параметры принимает шаблон -- следует выполнить **lxc-create --template <название шаблона> --help**:

```
lxc-create --template ubuntu --help
```

Например, скачивание и установка шаблона ubuntu может быть выполнена следующим образом:

```
lxc-create -t download -n ubuntu -- --dist ubuntu --release wily --arch amd64
или
lxc-create -n ubuntu -t ubuntu-cloud -- -r raring -T http://cloud-images.ubuntu.com/raring/current/raring-server-cloudimg-amd64-root.tar.gz
```

Запуск контейнера

Выполняется командой **lxc-start -n <название контейнера>**:

```
sudo lxc-start -n test_01
```

В данном случае будет запущен контейнер с ubuntu и мы сразу же попадаем в консоль этой системы. Как правило, логин root, а пароль генерируется случайный:

```
Root password is 'TfKDCy7K', please change !
```

Читать вывод на консоль в последней строке запуска контейнера (иногда логин и пароль root:root либо ubuntu:ubuntu,...). После входа рекомендуется его сменить командой passwd.

Согласно документации, отключиться от консоли можно комбинацией клавиш ctrl-a q (в некоторых случая, например при использовании терминальных менеджеров типа tmux или screen, может не работать! в этом случае нужно проверить тип терминала и попробовать с терминалом TERM=vt100).

Если запустить контейнеры с ключом `-d`, контейнер будет работать в фоновом режиме без подключения к консоли:

```
sudo lxc-start -d -n test_01
```

Просмотр запущенных контейнеров

```
sudo lxc-ls -f

NAME       STATE    IPV4       IPV6  AUTOSTART
----------------------------------------------
test_01    RUNNING  10.0.0.10  -     NO
```

Просмотр состояния конкретного контейнера

```
sudo lxc-info -n test_01

Name:  test_01
State: RUNNING
PID:   9172
IP:    10.0.0.10
CPU use: 164.34 seconds
Link:    veth-01
  TX bytes:    4.03 MiB
  RX bytes:    77.35 MiB
  Total bytes: 81.38 MiB
```

Подключение к консоли запущенного контейнера

Выполняется командой **lxc-console -n <название контейнера>**:

```
sudo lxc-console -n test-01
```

Для более комфортной работы, настроим доступ по ssh. Изначально командой lxc-console заходим на консоль контейнера и устанавливаем sshd (в случае выхода во внешнюю сеть, возможно потребуется скопировать содержимое /etc/resolv.conf из хост-машины), а также создаем нового пользователя user:

```
apt-get update && apt-get install ssh
adduser user
```

Впоследствии, для доступ к запущенному контейнеру достаточно:

```
ssh user@10.0.0.10
```

Остановка контейнера

Изнутри остановить контейнер можно обычным способом, через shutdown, poweroff или reboot, извне -- командой **lxc-stop -n <название контейнера>**:

```
sudo lxc-stop -n test-01
```

При этом контейнер нормально завершит свою работу (с сохранениями изменений во всех открытых файлах).

Клонирование контейнера

Рекомендуется использовать команду **lxc-clone -o <название базового контейнер> -n <название нового контейнера>**

```
sudo lxc-clone -o test-01 -n test-02
```

Хотя вместо lxc-clone можно воспользоваться обычным копированием каталогов (с последующим изменением файлов конфигурации - пути, hostname, MAC- и IP-адреса,...)

Настройка сети

Согласно документации, существует 5 способов виртуализации сети:

- empty
- phys
- veth
- macvlan
- vlan

loopback interface (type = empty)

Если в конфигурационном файле контейнера укажем настройки, наподобие:

```
lxc.network.type   = empty
lxc.network.hwaddr = 00:16:3e:67:4f:a5
lxc.network.flags  = up
```

То после старта, у виртуальной машины будет настроен только loopback interface:

```
NAME     STATE    IPV4  IPV6  AUTOSTART
-------------------------------------
test_01  RUNNING  -     -     NO
```

Использование физического интерфейса (type = phys)

Если конфигурационный файл контейнера привести к виду:

```
lxc.network.type = phys
lxc.network.link = eth1
lxc.network.name = eth0
```

то получим, по сути, проброс физического сетевого интерфейса (после старта контейнера, интерфейс eth1 исчезнет в хост-машине, а сеть перейдет под управление eth0 в виртуальной машине).

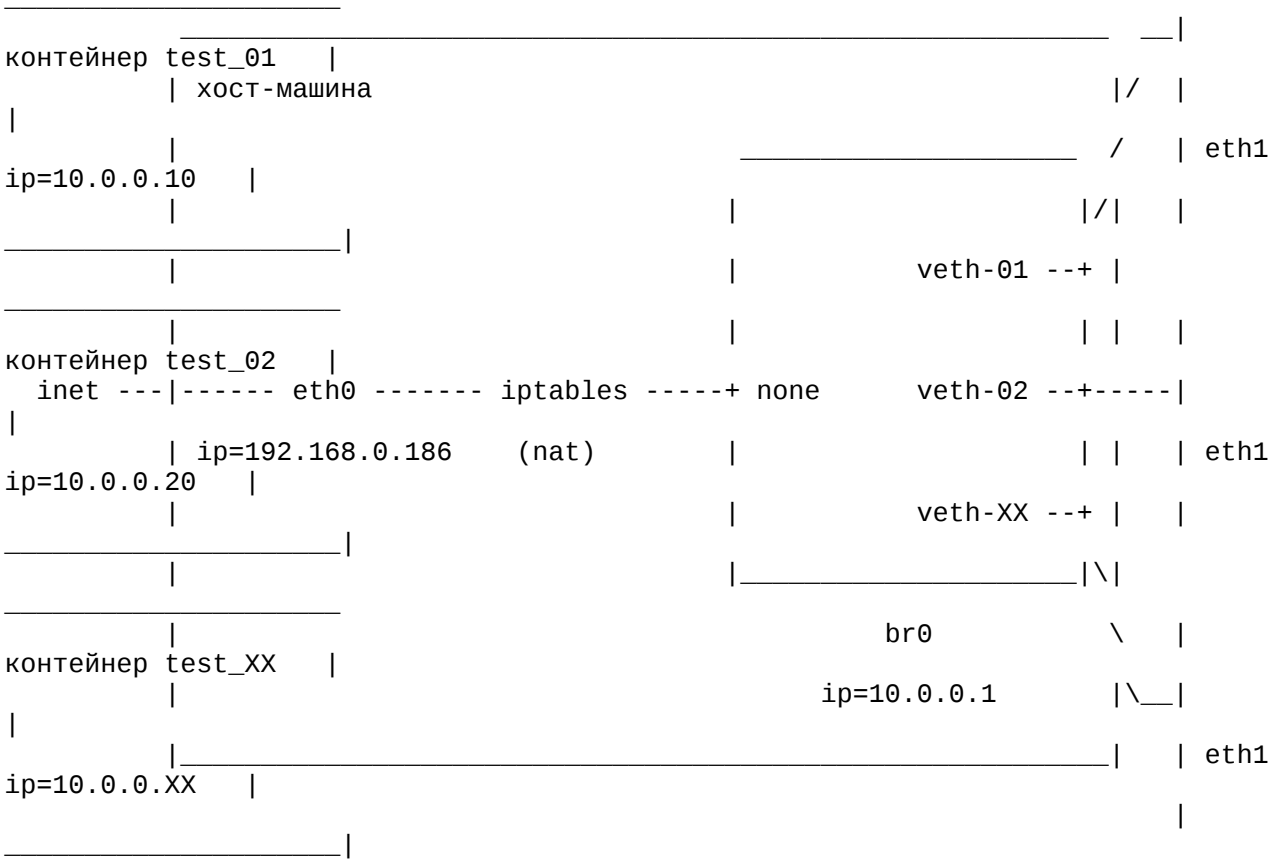
Использование сетевого стека хост-системы (type = veth)

Данная схема используется по умолчанию. При запуске контейнера с таким типом сети, на хост-машине создается специальный виртуальный интерфейс (в примере ниже, он называется veth-*). Этот виртуальный интерфейс фактически и использует контейнер для взаимодействия с внешней средой.

Рассмотрим несколько типовых случаев.

Трансляция ip-адресов

NAT уместно использовать в случае, когда на хост-машине имеется один статический ip (например, 192.168.0.186 на интерфейсе eth0) и несколько контейнеров, которым нужен выход в сеть через данный интерфейс. Условно, это можно проиллюстрировать следующей схемой:



Создадим сеть 10.0.0.0, в которой разместим виртуальные машины. Будем использовать пакет bridge-utils.

1. На хост-машине редактируем файл `/etc/network/interfaces`, дописывая блок настроек br0:

```
# Имеющиеся настройки lo и eth0 -- не трогаем

auto lo
```

```
iface lo inet loopback

auto  eth0
iface eth0 inet static
  address 192.168.0.186
  broadcast 192.168.0.191
  netmask 255.255.255.240
  gateway 192.168.0.190

# Создаем bridge br0

auto br0
 iface br0 inet static
 bridge_ports none
 bridge_fd 0
 address 10.0.0.1
 netmask 255.255.255.0
```

Для того, чтобы изменения были приняты, выполняем:

```
/etc/init.d/networking stop && /etc/init.d/networking start
```

Внимание! В случае каких-либо ошибок, может пропасть сеть.

2. Правим файл `/var/lib/lxc/test_01/config`, дописывая блок настроек сети:

```
lxc.utsname             = vm0               # имя хоста виртуальной машины
lxc.network.type        = veth              # тип сети, veth если
используется bridge
lxc.network.flags       = up                # поднимать сетевой интерфейс
при запуске системы
lxc.network.name        = eth1              # интерфейс внутри контейнера
lxc.network.link        = br0               # bridge, через который будет
работать виртуальный интерфейс
lxc.network.veth.pair   = veth-01           # имя сетевого адаптера для
этого контейнера на хостовой машине
lxc.network.ipv4        = 10.0.0.10/24      # сетевой адрес хоста
lxc.network.ipv4.gateway = 10.0.0.1         # шлюз
lxc.network.hwaddr      = 00:1E:2D:F7:E3:4E # мак-адрес хоста
```

3. Добавляем в таблицу nat правило:

```
iptables -t nat -A POSTROUTING -s 10.0.0.10/24 -j SNAT --to-source
192.168.0.186
```

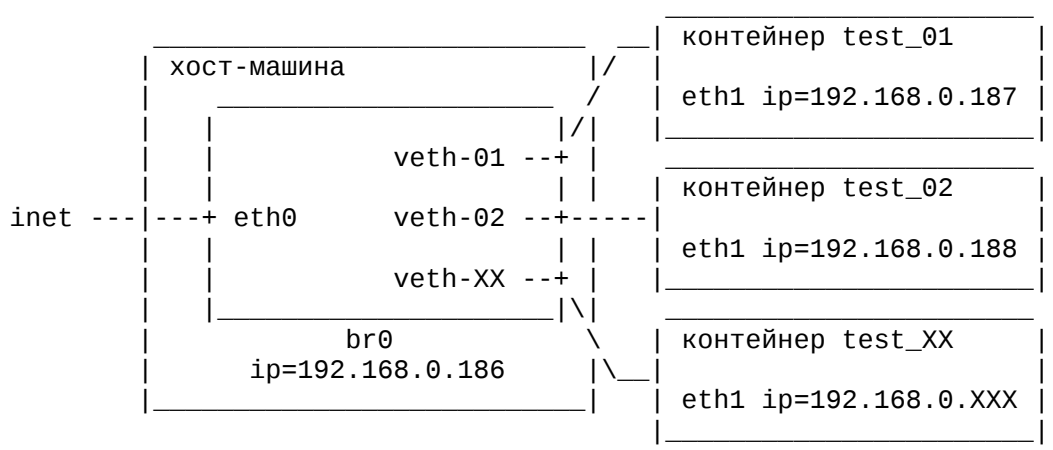
4. Запускаем контейнер test_01. В результате, данный хост должен получить при старте ip=10.0.0.10 и через шлюз 10.0.0.1, и далее через 192.168.0.186 -- выход в сеть.

Пункты 2-4 выполняем по аналогии для всех контейнеров сети.

Статические ip-адреса

Этот вариант уместно использовать, когда у каждого контейнера должен быть постоянный выделенный ip (и, соответственно, возможность заходить из инета внутрь контейнера напрямую).

Для настройки сети нужно создать bridge br0, который будет включать с одной стороны eth0 хост-машины, а с другой - виртуальные интерфейсы контейнеров:



1. На хост-машине отредактируем файл `/etc/network/interfaces`, дописывая блок настроек br0:

```
# Настройки lo -- не трогаем

auto lo
iface lo inet loopback

# Имеющуюся конфигурацию eth0 -- удаляем или комментируем

# auto  eth0
# iface eth0 inet static
#   address 192.168.0.186
#   broadcast 192.168.0.191
#   netmask 255.255.255.240
#   gateway 192.168.0.190

# Создаем bridge br0

auto br0
iface br0 inet static
   bridge_ports eth0
   bridge_fd 0
   address 192.168.0.186
   broadcast 192.168.0.191
   netmask 255.255.255.240
   gateway 192.168.0.190
```

Перегружаем сеть:

```
/etc/init.d/networking stop && /etc/init.d/networking start
```

2. Правим файл `/var/lib/lxc/test_01/config`, дописывая блок настроек сети:

```
lxc.utsname             = vm0               # имя хоста виртуальной машины
lxc.network.type        = veth              # тип сети, veth если
используется bridge
lxc.network.flags       = up                # поднимать сетевой интерфейс
при запуске системы
lxc.network.name        = eth1              # интерфейс внутри контейнера
lxc.network.link        = br0               # bridge, через который будет
работать виртуальный интерфейс
lxc.network.veth.pair   = veth-01           # имя сетевого адаптера для
этого контейнера на хостовой машине
lxc.network.ipv4        = 192.168.0.187/28  # сетевой адрес хоста
```

```
lxc.network.hwaddr       = 00:1E:2D:F7:E3:4E # мак-адрес хоста
lxc.network.ipv4.gateway = 192.168.0.190     # шлюз
```

В результате будем иметь следующую картину (на хост-машине):

```
br0   Link encap:Ethernet  HWaddr 00:16:3e:2f:f3:73
      inet addr:192.168.0.186  Bcast:192.168.0.191  Mask:255.255.255.240
      inet6 addr: fe80::216:3eff:fe2f:f373/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:437 errors:0 dropped:0 overruns:0 frame:0
      TX packets:367 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:0
      RX bytes:66408 (64.8 KiB)  TX bytes:119528 (116.7 KiB)
eth0  Link encap:Ethernet  HWaddr 00:16:3e:2f:f3:73
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:446 errors:0 dropped:0 overruns:0 frame:0
      TX packets:368 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:66872 (65.3 KiB)  TX bytes:119574 (116.7 KiB)
      Interrupt:25
veth-01  Link encap:Ethernet  HWaddr fe:de:ce:91:c1:81
      inet6 addr: fe80::fcde:ceff:fe91:c181/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:60 errors:0 dropped:0 overruns:0 frame:0
      TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:3840 (3.7 KiB)  TX bytes:6147 (6.0 KiB)
```

Результат вывода ifconfig внутри контейнера test_01:

```
eth1  Link encap:Ethernet  HWaddr 00:1e:2d:f7:e3:4e
      inet addr:192.168.0.187  Bcast:192.168.0.255  Mask:255.255.255.0
      inet6 addr: fe80::21e:2dff:fef7:e34e/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:51 errors:0 dropped:0 overruns:0 frame:0
      TX packets:39 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:3913 (3.8 KiB)  TX bytes:2742 (2.6 KiB)
```

Пункт 2 выполняем по аналогии для всех контейнеров сети.

Замечания:

- Контейнеры с такими параметрами смогут взаимодействовать с внешним миром напрямую, как отдельно стоящие машины.
- В идеале, вместо eth0 на хост-машине следует завести отдельный физический интерфейс для сетевого трафика контейнеров.

Использование DHCP

Сеть может быть настроена как статически в контейнерах, так и динамически и помощью DHCP-сервера, запущенного на хост-машине. Такой DHCP-сервер должен быть сконфигурирован для ответа на запросы на интерфейсе br0:

1. Устанавливаем и настраиваем DHCP-сервер

2. Файл `/etc/network/interfaces`, примет вид:

```
auto lo
iface lo inet loopback

auto br0
iface br0 inet dhcp
   bridge_ports eth0
   bridge_fd 0
   bridge_stp off
   bridge_maxwait 2
```

3. В файле `/var/lib/lxc/test_02/config`, удаляем или комментируем все статические настройки сети:

```
lxc.utsname             = vm0               # имя хоста виртуальной машины
lxc.network.type        = veth              # тип сети, veth если
используется bridge
lxc.network.flags       = up                # поднимать сетевой интерфейс
при запуске системы
lxc.network.name        = eth1              # интерфейс внутри контейнера
lxc.network.link        = br0               # bridge, через который будет
работать виртуальный интерфейс
```

Несколько виртуальных сетевых интерфейсов с разными MAC-адресами (type = macvlan)

При использовании схемы veth, все запущенные контейнеры, принадлежащие одной сети, видны и доступны друг-другу. Например, можно из хост-системы по ssh подсоединиться к машине 10.0.0.10, а из консоли этой машины -- попасть на консоль 10.0.0.20 и т.д.

Во избежание подобных ситуаций, т.е. в тех случаях, когда нужна полная изоляция контейнеров -- следует использовать схему macvlan.

Для демонстрации, на хост-машине редактируем файл `/etc/network/interfaces`, дописывая блок настроек br0:

```
# Имеющиеся настройки lo и eth0 -- не трогаем

auto lo
iface lo inet loopback

auto  eth0
iface eth0 inet static
  address 192.168.0.186
  broadcast 192.168.0.191
  netmask 255.255.255.240
  gateway 192.168.0.190

# Создаем bridge br0

auto br0
iface br0 inet static
  bridge_ports none
  bridge_fd 0
  address 10.0.0.1
  netmask 255.255.255.0
```

Файлы конфигурации каждого контейнера приводим к виду:

```
lxc.network.type         = macvlan
```

```
   lxc.network.macvlan.mode = vepa
   lxc.network.flags        = up
   lxc.network.name         = eth1
   lxc.network.link         = br0
   lxc.network.ipv4.gateway = 10.0.0.1
   lxc.network.ipv4         = 10.0.0.10/24  # тут каждому контейнеру присваиваем
собственный ip
```

После запуска контейнеров можно убедиться в их статусе:

```
  NAME     STATE    IPV4       IPV6  AUTOSTART
  ----------------------------------------------
  test_01  RUNNING  10.0.0.10  -     NO
  test_02  RUNNING  10.0.0.20  -     NO
```

При использовании macvlan следует указать один из режимов:

lxc.network.macvlan.mode = vepa
: В данном режиме, контейнеры ни из хост-системы ни между собой даже не пингуются

lxc.network.macvlan.mode = bridge
: Режим bridge создает особый мост (не то же самое, что стандартный Linux-bridge), который позволяет контейнерам общаться друг с другом, но изолирует их интерфейсы от хост-системы.

lxc.network.macvlan.mode = private
: Данный режим запрещает любую связь между LXC контейнерами.

Ещё про сеть:

- Configuration réseau de LXC
- Exploring LXC Networking
- Linux Containers and Networking
- 9.2.5 About Veth and Macvlan

Ограничение ресурсов

Память

Чтобы ограничить выделяемую контейнеру память, необходимо в его конфигурационном файле добавить следующий параметр:

```
   lxc.cgroup.memory.limit_in_bytes = 512M
   lxc.cgroup.memory.memsw.limit_in_bytes = 256M
```

Как вариант, можно те же действия выполнить из командной строки (перечень всех параметров -- см. в man lxc-cgroup):

```
   sudo lxc-cgroup -n tes_01 memory.limit_in_bytes 53687091
```

Более подробно, см.:

- Memory Resource Controller

CPU

Есть два параметра описывающих лимит процессора:

- lxc.cgroup.cpuset.cpus -- выделение конкретного ядра/ядер
- lxc.cgroup.cpu.shares -- приоритет (по умолчанию равно 1024)

В конфигурационном файле контейнера (выделяем под контейнер первых два ядра процессора):

```
lxc.cgroup.cpuset.cpus = 0,1
lxc.cgroup.cpu.shares = 512
```

Или то же из-под консоли:

```
sudo lxc-cgroup -n test_01 cpuset.cpus 0,1
sudo lxc-cgroup -n test_01 cpu.shares 512
```

Более подробно, см.:

- CPUSETS
- CFS Scheduler

Дисковые квоты: использование loopback-устройств

Если каталог контейнера будет размещён на отдельной файловой системе, её размер будет автоматически ограничен размером нижележащего блочного устройства. Файловая система может быть размещена на:

- файле (через loopback-устройство);
- томе LVM;
- томе btrfs;
- быть дисковым разделом;
- быть полноценным диском.

Последние два варианта, как правило, не используются из-за недостаточной их гибкости, хотя в некоторых случаях они вполне могут найти своё применение. Использование файлов считается более медленным чем непосредственное использование LVM или btrfs, поскольку вводит дополнительный слой (файловую систему, на которой располагается сам файл), с другой стороны этот способ является наиболее гибким.

Использование LVM и btrfs помимо прочих преимуществ обладает ещё тем, что позволяет создавать снэпшоты устройств, а в случае с btrfs ещё и снэпшоты со снэпшотов, что облегчает задачу клонирования контейнеров.

Ниже подробно рассматривается, как можно использовать файл для хранения файловой системы LXC.

Создаем файл определенного размера (в примере ниже он называется disk и имеет размер 600Мб), форматируем его и монтируем средствами хост-машины. А далее в этот каталог (/var/fs) устанавливаем контейнер:

```
mkdir /var/fs
dd if=/dev/zero of=/var/disk bs=1024 count=600000
mkfs.ext4 /var/disk
mount -t ext4 -o loop /var/disk /var/fs
lxc-create -t debian -n test_03 --lxcpath=/var/fs
```

Естественно, команду монтирования можно прописать в /etc/fstab. Ну и следует не забывать указывать ключ lxcpath при работе с контейнером, например:

```
lxc-start -n test_03 --lxcpath=/var/fs

lxc-ls -f --lxcpath=/var/fs

lxc-info -n test-03 --lxcpath=/var/fs

lxc-clone -o test-03 -n test-04 --lxcpath=/var/fs --newpath=/var/fs2
```

Запуск контейнеров поверх LVM

Принципиально данный способ не отличается от предыдущего: изначально подготавливается логический том требуемого размера, в который далее инсталлируется контейнер LXC.

Подробнее об использовании LVM: LVM.

Проверка ограничений

Для начала проверим, куда смонтирована cgroup (по умолчанию -- в /sys/fs/cgroup):

```
grep cgroup /proc/mounts
```

Данные по использованию памяти, процессора и пр., находятся в `/<точка монтирования cgroup>/lxc/<название запущенного контейнера>/<файл параметров>`:

```
cat /sys/fs/cgroup/lxc/test_01/memory.usage_in_bytes
```

Что произойдёт с процессом если он превысит отведённые ему лимиты памяти? Как проверить? Проверить можно очень просто, можно просто создать процесс, который запрашивает необходимое количество памяти:

```
#include <stdlib.h>
#include <stdio.h>

main()
{
#define SIZE_M 100
#define SIZE SIZE_M*1024*1024
    void * a = malloc(SIZE);

    char ch;
    read(0, &ch, 1);
}


$ gcc -o /tmp/1 /tmp/1.c
$ /tmp/1
(программа ждёт пока будет нажато ctrl-d, но пока не надо нажимать)
```

В другом окне:

```
$ ps aux | grep /tmp/1
igor     30940  0.0  0.0 104176   332 pts/2    S+   13:44   0:00 /tmp/1
igor     30970  0.0  0.0   6036   900 pts/3    S+   13:44   0:00 grep /tmp/1
```

Видно, что программа занимает чуть больше запрошенных 100 мегабайтов (пятая колонка, первый ряд).

Проброс устройств

```
  sudo lxc-device add -n test_01 /dev/ttyUSB0 /dev/ttyS0
```

Конфигурация контейнера

Конфигурация контейнера находится в файле

```
 /var/lib/lxc/НАЗВАНИЕ-КОНТЕЙНЕРА/config
```

Конфигурация контейнера описывает всевозможные аспекты его существования, начиная от доменного имени, IP-адреса и заканчивая ограничениями всех ресурсов, которые он использует.

Подробнее:

- lxc.container.conf(5)

X-Server

Устанавливаем внутри контейнера графическую оболочку и софт для работы с удаленным рабочим столом по протоколу RDP:

```
  apt-get install xorg lxde xrdp
```

Если сеть настроена для работы с NAT, то на хосте приводим правила iptables к следующему виду (10.0.0.100 -- ip контейнера, 192.168.0.186 -- статический ip хост-машины, 3389 -- порт по умолчанию для работы xrdp):

```
  iptables -t nat -A POSTROTING -s 10.0.0.100 -j MASQUERADE
  iptables -t nat -A PREROUTING -d 192.168.0.186 -p tcp -m tcp --dport 3389 -j
DNAT --to-destination 10.0.0.100:3389
```

В Windows заходим в «Пуск» -> «Все программы» -> «Стандартные» -> «Подключение к удаленному рабочему столу», вводим ip хост-машины; во время подключения -- выбираем модуль «sesman-Xvnc» и вводим логин/пароль пользователя контейнера.

LXC без LXC

Функционал LXC фактически представлен механизмами ядра Linux и утилиты lxc, работающие в пространстве пользователя (userspace). Это означает, что контейнеры LXC в

полной мере могут использоваться и без утилит lxc. Естественно, что в этом случае потребуется какая-то другая программа/система, выполняющая управление ими.

libvirt

libvirt LXC-драйвер не использует никакие утилиты LXC и никак не зависит от них.

Возможности ядра, необходимые для работы LXC:

- control groups (требуемые контроллеры: cpuacct, memory, devices; рекомендуемые: cpu, freezer, blkio);
- namespaces (требуются mount, ipc, pid и uts; если используется собственная сеть в контейнере, то ещё требуется net).

Подробнее о возможностях драйвера контейнеров LXC библиотеки libvirt:

- LXC container driver (англ.)

libcontainer

LXC в сравнении с другими проектами

LXC vs OpenVZ

Хотя и немного устаревшую, но всё же полезную информацию на эту тему можно найти здесь:

- OpenVZ vs LXC (рус.), сравнение от менеджера проекта OpenVZ, Кира Колышкина
- Устарел ли OpenVZ? (рус.), попытка убедить читателя, что OpenVZ не устарел